

AWiR – ćwiczenie 5

Temat:

Spring Security – ochrona widoków (MVC) i REST, własna strona logowania + Remember-me

Cel ćwiczenia:

Rozszerz aplikację z **Ćwiczenia 4 (REST)** o **Spring Security**:

- zabezpiecz **widoki MVC** (/users/**) oraz **REST API** (/api/**),
- dodaj **własną stronę logowania** (/login) z checkboxem „remember me”,
- włącz **role** USER i ADMIN,
- skonfiguruj wyjątki (dostęp zabroniony) i strony publiczne (login, assets, H2, Swagger UI).

Niezbędne narzędzia:

1. DE: STS 4 / IntelliJ / VS Code
2. Maven 3.9+
3. JDK 21
4. Spring Boot 3.3.x
5. Baza: H2 (z ćw. 3)

Dotychczasowy projekt zawierał:

- Masz już projekt z ćw. 3-4:
- @Entity User, repozytorium i serwis,
- kontroler MVC w edu.zut.awir.web.mvc.UserController (Thymeleaf),
- kontroler REST w edu.zut.awir.web.rest.UserRestController (JSON),
- H2 + JPA w application.properties,
- Swagger UI (springdoc) dla REST.

W tym ćwiczeniu dodaj:

- zależność **Spring Security** w pom.xml,
- konfigurację SecurityConfig (bean SecurityFilterChain, PasswordEncoder, UserDetailsService),
- **własny widok logowania** login.html z checkboxem remember-me,
- reguły autoryzacji dla /users/** (MVC) i /api/** (REST),
- **CSRF**: włączone dla MVC, **wyłączone dla /api/**** (ułatwia testy Postman/curl),
- Remember-me z kluczem aplikacyjnym.

Nowa struktura projektu

```
src/main/java/edu/zut/awir/
├── AwirApplication.java
├── config/
│   └── SecurityConfig.java # NOWE: konfiguracja Spring Security 6
├── model/
│   └── User.java
├── repository/
│   └── UserRepository.java
├── service/
│   └── UserService.java
├── web/
│   ├── mvc/
│   │   ├── UserController.java # MVC (HTML)
│   │   └── LoginController #NOWE: kontroler logowania
│   └── rest/
│       ├── UserRestController.java # REST (JSON)
│       └── RestExceptionHandler.java
└── src/main/resources/
    ├── application.properties
    └── templates/
        ├── add-user.html
        ├── edit-user.html
        ├── user-info.html
        ├── user-list.html
        └── login.html # NOWE: własna strona logowania z remember-me
```

1. Rozszerzenie zależności w pliku pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

2. Konfiguracja bezpieczeństwa

src/main/java/edu/zut/awir/config/SecurityConfig.java

```
package edu.zut.awir.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.util.matcher.AntPathRequestMatcher;

@Configuration
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
```

```

UserService users(PasswordEncoder encoder) {
    UserDetails user = User.withUsername("user")
        .password(encoder.encode("user123"))
        .roles("USER")
        .build();
    UserDetails admin = User.withUsername("admin")
        .password(encoder.encode("admin123"))
        .roles("ADMIN")
        .build();
    return new InMemoryUserDetailsManager(user, admin);
}

```

@Bean

```

SecurityFilterChain http(HttpSecurity http) throws Exception {
    http
        // CSRF: włączone domyślnie, ale ignorujemy REST
        .csrf(csrf -> csrf.ignoringRequestMatchers("/api/**", "/h2/**"))

        // autoryzacja żądań
        .authorizeHttpRequests(reg -> reg
            .requestMatchers(
                "/", "/login", "/css/**", "/js/**", "/images/**",
                "/swagger-ui/**", "/v3/api-docs/**", "/h2/**"
            ).permitAll()
            .requestMatchers("/users/**").hasAnyRole("USER", "ADMIN") // MVC
            .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN") // REST API
            .anyRequest().authenticated()
        )

        // logowanie formularzowe z własnym widokiem
        .formLogin(form -> form
            .loginPage("/login")
            .loginProcessingUrl("/login")
            .defaultSuccessUrl("/users/list", true)
            .failureUrl("/login?error")
            .permitAll()
        )

        .httpBasic(basic -> {})

        // remember-me
        .rememberMe(rm -> rm
            .rememberMeParameter("remember-me")
            .key("awir-remember-me-secret-key")
            .tokenValiditySeconds(60 * 60 * 24 * 14) // 14 dni
        )

        // wylogowanie
        .logout(logout -> logout
            .logoutRequestMatcher(new AntPathRequestMatcher("/logout"))
            .logoutSuccessUrl("/login?logout")
            .deleteCookies("JSESSIONID", "remember-me")
            .permitAll()
        )

        // H2 console (frames)
        .headers(h -> h.frameOptions(f -> f.disable()));

    return http.build();
}

```

}

3. Własna strona logowania (Thymeleaf)

src/main/resources/templates/login.html

```
<!DOCTYPE html>
<html lang="pl" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Logowanie</title>
<style>
body { font-family: system-ui, sans-serif; max-width: 420px; margin: 5rem
auto; }
label { display:block; margin:.5rem 0 .25rem; }
input[type=text], input[type=password] { width:100%; padding:.5rem; }
.error { color:#b00020; margin:.5rem 0; }
</style>
</head>
<body>
<h2>Logowanie</h2>
<form th:action="@{/login}" method="post">
<div th:if="${param.error}" class="error">Niepoprawny login lub hasło.</div>
<div th:if="${param.logout}">Wylogowano pomyślnie.</div>

<label for="username">Użytkownik</label>
<input id="username" name="username" type="text" required />

<label for="password">Hasło</label>
<input id="password" name="password" type="password" required />

<label style="display:flex; align-items:center; gap:.5rem; margin-top:.5rem;">
<input type="checkbox" name="remember-me" /> Zapamiętaj mnie
</label>

<p style="margin-top:1rem;">
<button type="submit">Zaloguj</button>
</p>
</form>
</body>
</html>
```

4. Kontroler logowania

```
@Controller
public class LoginController {
    @GetMapping("/login")
    public String loginPage() { return "login"; }
}
```

5. Konfiguracja aplikacji

```
# Spring Security - strona logowania domyślnie /login (obsłużona w
SecurityConfig)
server.error.whitelabel.enabled=false
```

6. Działanie

- MVC (Thymeleaf): Wejście na /users/list bez sesji → przekierowanie na /login. Po poprawnym logowaniu (np.

user/user123) → dostęp do widoków. Zaznaczenie „Zapamiętaj mnie” ustawi cookie remember-me (14 dni).

- REST: Wejście przez HTTP Basic (np. Postman/curl) – -u user:user123.
- H2 (/h2) i Swagger UI są dostępne bez logowania.

7. Zadania do wykonania

- Ogranicz REST: GET dla USER, a POST/PUT/DELETE tylko dla ADMIN.
- Dodaj stronę **/access-denied** i `exceptionHandling().accessDeniedPage(...)`.

8. Zdania dodatkowe

- Zastąp `InMemoryUserDetailsManager` na **JDBC** z tabelami Spring Security (users, authorities) w H2
- Dodaj prosty logout link na navbarze widoków MVC.